



Git & Version Control Basics

Khin Sandar Kyaw, PhD

(Pronouns: she/her/hers)

Visiting Faculty

Faculty of Mathematics and Science

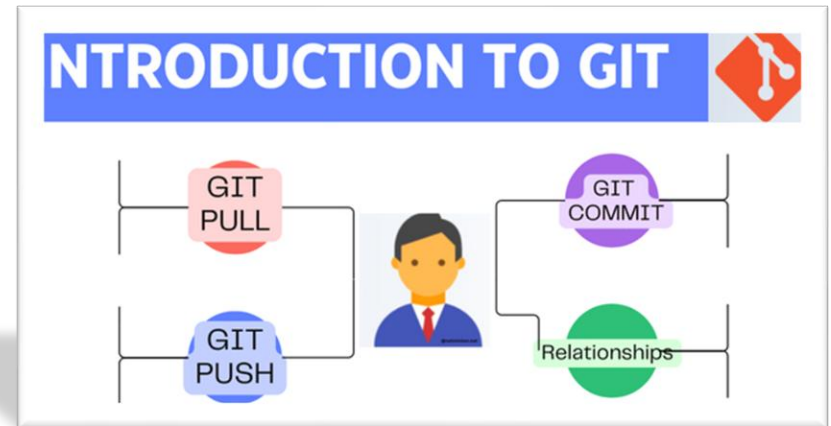
PARAMI UNIVERSITY

1. Introduction to Version Control

- Version Control System (VCS): Software tools that help manage changes to source code over time.
- Why use VCS?
 - Tracks and revert changes in code
 - Enables collaboration
 - Branching and merging
 - Backup and restore
 - Maintains project history

2. What is Git?

- **Git:** A distributed version control system created by Linus Torvalds in 2005.
- **Distributed:** Each developer has a full copy of the repository.
- **Popular Hosting Services:**
 - GitHub
 - <https://github.com/>
 - GitLab
 - <https://about.gitlab.com/>
 - Bitbucket
 - <https://bitbucket.org/>



3. Git Basics

- 3.1 Git Workflow
- 3.2 Common Git Commands

3.1 Git Workflow

- **Working Directory** – Where files are edited.
- **Staging Area (Index)** – Where changes are prepared.
- **Repository (.git directory)** – Where committed changes are stored.

3.2 Common Git Commands

Command

- `git init`
- `git clone <url>`
- `git status`
- `git add <file>`
- `git commit -m "msg"`
- `git log`
- `git diff`
- `git branch`
- `git checkout <branch>`
- `git merge <branch>`
- `git pull`
- `git push`

Description

Initialize a new Git repository

Create a local copy of a remote repo

Show changes and staging info

Add file to staging area

Commit changes to the repo

Show commit history

Show differences between versions

List or create branches

Switch to another branch

Merge a branch into current one

Fetch and merge from remote repo

Upload local commits to remote repo

4. Branching and Merging

- **Branching:** Allows development of features independently.
- **Main branch:** Often named main or master.
- **Merging:** Combines changes from different branches.
 - Fast-forward merge
 - Three-way merge
 - Merge conflicts may require manual resolution.

5. Remote Repositories

- **Remote Repository:** Hosted copy of the project on platforms like GitHub.
- **Key Commands:**
 - `git remote add origin <url>` – Add remote repository
 - `git push -u origin main` – Push first commit to remote
 - `git pull origin main` – Update from remote

6. Good Practices

- Commit often, with clear messages
- Use `.gitignore` to exclude unnecessary files
- Pull before you push to avoid conflicts
- Use branches for features or fixes
- Write informative commit messages

7. .gitignore File

- Specifies intentionally untracked files to ignore.

Example:

```
__pycache__/
```

```
*.log
```

```
*.tmp
```

```
.env
```

```
node_modules/
```

8. Git vs Other VCS (Version Control System)

Feature	Git	SVN (Centralized) [Subversion]
Type	Distributed	Centralized
Offline work	Yes	No
Performance	Fast	Slower
Branching	Lightweight	Heavyweight

Best For

Individual or small teams
Large, asset-heavy games or enterprises
Legacy enterprise projects
Simple DVCS with easier CLI

Recommended VCS

Git
Perforce
SVN
Mercurial

9. Git: Advantages and Disadvantages

- Offline commits & history
- Efficient branching/merging
- Widespread support & tools
- Open-source and free
- Steeper learning curve (commands, concepts)
- History rewriting can be dangerous if misused

10. Advanced Git (Intro Only)

- **Rebase:** Streamlines a feature branch's history
- **Stash:** Temporarily store changes
- **Tags:** Mark important commits (e.g., releases)

11. Summary

- Git is essential for modern software development.
- Learn core concepts: init, add, commit, push, pull.
- Practice branching and merging.
- Use **GitHub** for collaboration and hosting.

12. Further Reading/ Practice

- [Pro Git Book \(free online\)](#)
- [GitHub learning Lab](#)



Git Installation

Objectives

By the end of this lecture, students will be able to:

- Understand system requirements for Git
- Install Git on Windows, macOS, and Linux
- Verify the installation
- Set up user configuration for Git

1. System Requirements

- Git works on:
 - **Windows** (7 or later)
 - **macOS** (10.9 or later)
 - **Linux** (any modern distribution)
- Requires basic command line access (Command Prompt, Terminal, Bash)

2. Installing Git

- 2.1 Windows
- 2.2 macOS
- 2.3 Linux

2.1 Windows

1. Visit: <https://git-scm.com>
2. Click **Download for Windows**
3. Run the installer (.exe)
4. Recommended Settings:
 - Choose “Git from the command line and also from 3rd-party software”
 - Use default editor (or change to VS Code)
 - Use Git from Git Bash and Command Prompt
 - Use default SSH and line ending options
5. Finish setup and open **Git Bash**

2.2 mac OS

- Option A: Homebrew (recommended)
 - `brew install git`
- Option B: Xcode Command Line Tools
 - `xcode-select --install`
- Option C: Download from git-scm.com

2.3 Linux

- Use the package manager:
 - **Debian/Ubuntu:**
 - `sudo apt update`
 - `sudo apt install git`
 - **Fedora:**
 - `sudo dnf install git`
 - **Arch Linux:**
 - `sudo pacman -S git`

3. Verify Installation

- Run the following command in Terminal or Git Bash:
 - `git --version`
- Expected output:
 - `git` version 2.x.x

4. Git Basic Configuration

- After installation, configure your identity:
 - `git config --global user.name "Your Name"`
 - `git config --global user.email "you@example.com"`
- **To confirm:**
 - `git config --list`

5. Optional: Text Editor Configuration

- To use VS Code or another editor by default:
 - `git config --global core.editor "code --wait"`

6. Optional: Generate SSH Key for GitHub

- For pushing to GitHub via SSH:
 - `ssh-keygen -t ed25519 -C "you@example.com"`
- Press Enter to save to the default path
- Use `cat ~/.ssh/id_ed25519.pub` to copy the key
- Add the key to GitHub under **Settings > SSH and GPG keys**

7. Summary

- Git can be installed on any major OS via official website or package managers.
- Post-install setup involves setting username, email, and optionally your preferred editor.
- Verifying the installation ensures Git is ready for use.

Next Steps

- Try cloning a test repository
- Create your first Git project locally
- Practice basic Git commands: `git init`, `git add`, `git commit`

Hands-On with Git

1. Clone a Test Repository (Remote → Local)
2. Create Your First Git Project Locally
3. Practice Basic Git Commands

1. Clone a Test Repository (Remote → Local)

- Try cloning a public Git repository from GitHub:
 - `git clone https://github.com/git-guides/git-sample-project.git`
 - `cd git-sample-project`
- Explore it:
 - `ls` *# List files*
 - `git log` *# View commit history*

2. Create Your First Git Project Locally

- Set up your own Git project from scratch:
 - `mkdir my-first-git-project`
 - `cd my-first-git-project`
 - `git init`
- Output
 - Initialized `empty` Git repository in `/your/path/my-first-git-project/.git/`
- Create a file and make your first commit:
 - `echo "# My First Git Project" > README.md`
 - `git add README.md`
 - `git commit -m "Initial commit: add README"`
- Check the status:
 - `git status`
 - `git log`

Summary of Commands

Command

- `git clone <url>`
- `git init`
- `git add <file>`
- `git commit -m "msg"`
- `git status`
- `git log`

Purpose

Copy remote repo to your machine

Start a new Git repo locally

Stage changes

Save staged changes

Check current state

View commit history

3. Practice Basic Git Commands

Command

- `git init`
- `git add <file>`
- `git commit -m "message"`
- `git status`
- `git log`
- `git log --oneline --graph`

What it does

Initialize a new Git repo

Stage a file for commit

Save a snapshot of your project

Check what's changed and staged

View commit history

View a visual history tree



GitHub

Push a Local Git Project to GitHub

What is GitHub?

- GitHub is a **web-based platform** for **hosting**, **managing**, and **collaborating** on code using the **Git version control system**.

Key Features of GitHub

Feature

Description

 **Version Control**

GitHub uses Git to track changes in your code, so you can revert, branch, and collaborate safely.

 **Collaboration**

Teams can work together on code, review each other's changes (pull requests), and resolve issues.

 **Repositories**

A GitHub **repository** (or "repo") is where your project lives — it includes files, history, and settings.

 **Remote Hosting**

GitHub acts as a **remote** for your local Git repository, allowing backups and team access.

 **Open Source & Private Projects**

Host public open-source projects or private company/team codebases.

 **Tools & Integration**

Supports continuous integration (CI/CD), project boards, actions, wikis, and more.

GitHub vs Git

Git

- A **command-line tool**
- Installed on your computer
- Stores code locally
- You manage changes

GitHub

- A **cloud platform** that uses Git
- Accessed via browser or Git tools
- Stores your repo **remotely** on GitHub servers
- GitHub adds tools for **teamwork and sharing**

1. Create a GitHub Account

- Go to <https://github.com>
- Sign up or log in

Example use Case

- Create a local Git repo
- Push it to GitHub
- Collaborate with classmates or coworkers
- Submit pull requests, review code, and track issues

2. Create a New Repository on GitHub

1. Click the + icon (top right) → **New repository**
2. Fill in:
 - **Repository name:** e.g., `my-first-github-repo`
 - Optionally add description
 - Choose **Public** or **Private**
 - **Do NOT initialize** with README (you already have one locally)
3. Click **Create repository**

3. Link Your Local Project

- If you already have a Git project locally:
 - `cd my-first-git-project`
- Add GitHub as the remote:
 - `git remote add origin https://github.com/yourusername/my-first-github-repo.git`
- Check it worked:
 - `git remote -v`

4. Push to GitHub

- First push with upstream tracking:
 - `git push -u origin main`
- ! If your branch is named master, use master instead of main.
- You may be prompted to log in using a GitHub token (personal access token), not your password.

Optional: Use SSH instead of HTTPS

1. Generate SSH key:

- `ssh-keygen -t ed25519 -C "your_email@example.com"`

2. Copy your public key:

- `cat ~/.ssh/id_ed25519.pub`

3. Add it in **GitHub > Settings > SSH and GPG Keys > New SSH Key**

4. Switch remote to SSH:

- `git remote set-url origin git@github.com:yourusername/my-first-github-repo.git`

Now you can push without typing credentials every time!

Recap of Commands

Command

- `git remote add origin <url>`
- `git push -u origin main`
- `git remote -v`
- `git log`
- `git status`

Purpose

Add GitHub repo as remote

Push your local commits to GitHub

Verify remote repository URLs

View commit history

View working directory status

Example Workflow

```
mkdir my-first-git-project
```

```
cd my-first-git-project
```

```
git init
```

```
echo "# My Project" > README.md
```

```
git add README.md
```

```
git commit -m "Initial commit"
```

```
git remote add origin https://github.com/yourusername/my-first-github-repo.git
```

```
git push -u origin main
```

In-Class Activity

- Initialize repo
- Commit file
- Push to GitHub
- Clone a peer repo

Example

1. `$ git config --global user.name "Your Name"`
2. `$ git config --global user.email xxx@gmail.com`
3. `$ mkdir my-project`
4. `$ cd my-project`
5. `$ git init`
6. `$ echo "#My Project" > README.md`
7. `$ git status`
8. `$ git add README.md`
9. `$ git add`
10. `$ git commit -m "Add initial README file"`
11. `$ git log`